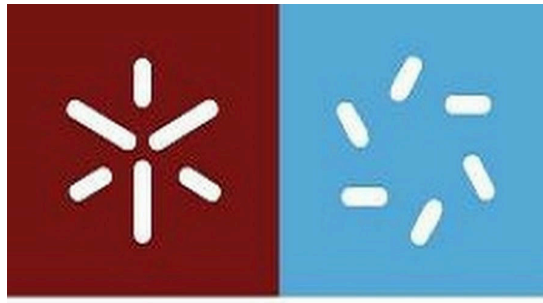




Universidade do Minho
Escola de Ciências

Computação Gráfica

2024/2025

Trabalho Prático

Fase 4 - Normais e Coordenadas de Texturas

Grupo 12

Afonso Martins n°A102940

Luis Silva n° A105530

Pedro Gomes n° A102929

Ricardo Vilaça n°A102879

Contents

1. Introdução	3
2. Alterações ao gerador	3
2.1. Cálculo das normais	4
2.1.1. Plano	4
2.1.2. Box	4
2.1.3. Sphere	4
2.1.4. Cone	4
2.1.5. Bezier	4
2.1.6. Torus	5
2.2. Cálculo das coordenadas das texturas	5
2.2.1. Plano	5
2.2.2. Box	5
2.2.3. Sphere	5
2.2.4. Cone	5
2.2.5. Superfícies de bezier	6
2.2.6. Torus	6
3. Alterações à engine	6
3.1. Alterações à estrutura de dados	6
3.2. Parser do ficheiro 3d	7
3.3. Setup das luzes	8
3.4. Setup das cores	8
4. Sistema Solar	8
5. Conclusão	8

1. Introdução

Nesta última fase do projeto, alterou-se tanto o gerador como a engine de forma a poderem ser definidas fontes de luz presentes no espaço e para se definir as cores e ou texturas que um modelo possa ter.

Para tal, o gerador é agora capaz de criar as coordenadas de posição, o vetor da normal e as coordenadas de textura para cada ponto da primitiva criada. A engine foi também alterada para poder interpretar estes novos valores para funcionalidades de iluminação e texturas. A engine é também capaz de interpretar novos campos presentes no xml, entre eles as **lights**, **texture** e **color**. Por fim criou-se um modelo do sistema solar com as devidas texturas para usar como exemplo.

Ao longo deste relatório explicaremos a forma como foi implementada cada uma das novas funcionalidades.

2. Alterações ao gerador

Como foi explicado em cima, o gerador foi alterado de forma a gerar, para além das coordenadas de posição, as coordenadas de textura e o vetor normal de cada ponto. Para tal efeito, alterou-se o formato dos ficheiros onde são guardados os modelos.

O seu formato ficou então:

- A primeira linha continua a dizer qual a primitiva representada neste ficheiro
 - De seguida fica definida cada componente da primitiva, começando por definir qual primitiva do GLUT a usar, seguido de todos os pontos que a compõe no formato
 - x position; x normal; x texture
 - y position; y normal; y texture
 - z position; z normal
- acabando com a escrita de um **GL_END**

```
1 plane
2 GL_TRIANGLE_STRIP
3 1;0;1
4 0;1;0
5 1;0
6 1;0;1
7 0;1;0.333333
8 0.333333;0
9 0.333333;0;0.666667
10 0;1;0
11 1;0
12 0.333333;0;0.666667
13 0;1;0.333333
14 0.333333;0
15 -0.333333;0;0.333333
16 0;1;0
17 1;0
18 -0.333333;0;0.333333
19 0;1;0.333333
20 0.333333;0
21 -1;0;0
22 0;1;0
23 1;0
24 -1;0;0
25 0;1;0.333333
26 0.333333;0
27 GL_END
```

Figure 1: Formato do ficheiro .3d

2.1. Cálculo das normais

2.1.1. Plano

Como se supõe que o plano esteja no plano XZ, orientado no sentido positivo em relação ao eixo do Y, temos que a normal para todos os pontos do plano é (0,1,0).

2.1.2. Box

O cálculo das normais da box é semelhante ao do plano, sendo a box definida por 6 planos do mesmo tamanho. Como tal, existem 6 normais referentes a estes 6 planos, sendo elas:

- $\vec{n}_1 = (0,1,0)$ -> face no sentido positivo do eixo Y
- $\vec{n}_2 = (0,-1,0)$ -> face no sentido negativo do eixo Y
- $\vec{n}_3 = (-1,0,0)$ -> face no sentido negativo do eixo X
- $\vec{n}_4 = (1,0,0)$ -> face no sentido positivo do eixo X
- $\vec{n}_5 = (0,0,-1)$ -> face no sentido negativo do eixo Z
- $\vec{n}_6 = (0,0,1)$ -> face no sentido positivo do eixo Z

2.1.3. Sphere

A normal de uma esfera num dado ponto da superfície é igual ao vetor normalizado que vai do centro da esfera até a esse ponto.

2.1.4. Cone

O cálculo das normais para os pontos do cone está dividido em duas etapas: o cálculo das normais da base e o cálculo das normais das faces laterais. Para todos os pontos da base, a normal terá coordenadas (0, -1, 0), visto que a base do cone deve estar no plano XZ e, no total do cone, a base está virada para baixo.

Na obtenção dos vetores normal das faces laterais do cone, as componentes x e z do vetor normal são iguais ao valores de x e z dos pontos onde são calculadas, sendo possível obtê-las através das seguintes equações:

- $x = \sin(i * \text{angleStep} * \pi/180)$
- $z = \cos(i * \text{angleStep} * \pi/180)$
- $\text{angleStep} = 360/\text{slices}; i = \text{slice atual}$

Para o cálculo da componente y da normal usamos as seguintes equações:

- $y = \sin(\beta)$
- $\beta = 180 - \alpha$
- $\alpha = \text{atan}(\text{height}/\text{radius})$

2.1.5. Bezier

Para a criação da superfície cúbica de Bezier, é necessário calcular os pontos que a compõe usando a seguinte equação:

$$\frac{\partial p(u,v)}{\partial u} = UD * B * P * B^T * V$$
$$\frac{\partial p(u,v)}{\partial v} = U * B * P * B^T * VD$$

Sendo $U = [u^3 \ u^2 \ u \ 1]$, $UD = [3u^2 \ 2u \ 1 \ 0]$, $B = \begin{bmatrix} -1 & 3 & -3 & 1 \\ 3 & -6 & 3 & 1 \\ -3 & 3 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix}$, $P = \begin{bmatrix} P_{00} & P_{01} & P_{02} & P_{03} \\ P_{10} & P_{11} & P_{12} & P_{13} \\ P_{20} & P_{21} & P_{22} & P_{23} \\ P_{30} & P_{31} & P_{32} & P_{33} \end{bmatrix}$,

$$B^T = \begin{bmatrix} -1 & 3 & -3 & 1 \\ 3 & -6 & 3 & 1 \\ -3 & 3 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix}, V = \begin{bmatrix} v^3 \\ v^2 \\ v \\ 1 \end{bmatrix} \text{ e } VD = \begin{bmatrix} 3v^2 \\ 2v \\ 1 \\ 0 \end{bmatrix}$$

Dados os valores de u e v , a normal de qualquer ponto é o resultado do produto interno dos vetores tangentes ao ponto, cujo cálculo se encontra a seguir:

- $\vec{u} = \frac{\partial p(u,v)}{\partial u}$
- $\vec{v} = \frac{\partial p(u,v)}{\partial v}$
- $\vec{n} = \vec{u} \cdot \vec{v}$

O vetor $\vec{n}$ representa, assim, o vetor normal para um dado ponto (u, v) .

2.1.6. Torus

Os vetores normal para pontos num torus são calculados assumindo uma lógica semelhante à usada para esferas, sendo neste caso usado o centro da slice em que se está a trabalhar invés do centro do torus.

2.2. Cálculo das coordenadas das texturas

2.2.1. Plano

Sendo a textura definida num espaço 2D no formato de um plano, para associar a textura ao plano basta dividir a textura pelo mesmo número de divisões que compõe o plano e associar cada ponto do plano ao respetivo ponto na textura. Como se começa a contruir o plano no canto inferior direito, começamos também a contar a textura no ponto $(s = 1, t = 0)$, avançando o t em 1/divisões com cada linha construída e o diminuindo o s em 1/divisões com cada coluna da linha atual construída.

2.2.2. Box

Como a box é construída com 6 planos, o cálculo das coordenadas da textura é igual, tendo só que ter cuidado na orientação da textura em cada face da box.

2.2.3. Sphere

O cálculo das coordenadas de textura da sphere é semelhante ao do plano, no entanto a textura é dividida no eixo t pelo número de stacks da sphere e dividida no eixo s pelo número de slices da sphere. Assim podemos associar cada quadrado da textura ao ponto respetivo.

No entanto como o topo da esfera é só um ponto, a sua coordenada de textura é igual a $((\text{slice atual} + 0.5) * 1/\text{slices})$.

2.2.4. Cone

Para o cone, o cálculo é também dividido em duas partes, a base e as laterais. Para a base, definimos que todos os pontos menos o central têm coordenadas de textura $(s = 0, t = 0)$, enquanto o central tem coordenadas $(s = 0, t = 1)$;

Para as laterais, como cone é construído do topo para baixo, começamos a percorrer a textura do canto superior esquerdo. O vértice de cada face do cone tem coordenadas $(s = i * \text{textureSliceStep}, t = 1)$, já os outros pontos de cada face tem coordenadas $(s = i * \text{textureSliceStep}, t = 1 - j * \text{textureStackStep})$ sendo:

- $i = \text{current slice}$
- $j = \text{current stack}$
- $\text{textureSliceStep} = 1/\text{slices do cone}$

- textureStackStep = 1/stack do cone

2.2.5. Superfícies de bezier

Como as patches de bezier estão já subdivididas, basta associar o **u** e o **v** do ponto atual ao **s** e **t** respetivamente.

2.2.6. Torus

As coordenadas de textura do torus são calculadas de maneira a que a textura se repita em cada slice, definindo um lado da slice com $t = 0$ e a outra com $t = 1$. De maneira a que a textura seja igual em cima do torus e em baixo, o **s** é calculado de acordo com a stack atual, sendo que se a stack atual for maior que metade do número de stacks, o **s** começa a decrescer criando assim um reflexo da textura.

3. Alterações à engine

Nesta fase foi necessário, para além de alterar estruturas dentro da engine, alterar a forma como se lê os XML para se poder ler as novas tags *lights*, *color* e *texture*.

3.1. Alterações à estrutura de dados

Para além de alterar as estruturas *File* e *Model*, criou-se também duas novas estruturas de dados: *Color* que descreve a cor de um modelo e é composta por 15 atributos,

```
struct Color {
    float diffuseR = 200, diffuseG = 200, diffuseB = 200;
    float ambientR = 50, ambientG = 50, ambientB = 50;
    float specularR = 0, specularG = 0, specularB = 0;
    float emissiveR = 0, emissiveG = 0, emissiveB = 0;
    float shininess = 0;
};
```

Figure 2: Estrutura de dados 'Color'

e *Light* que descreve uma fonte de luz no espaço.

```
struct Light {
    LightType type;
    float pos[3];
    float dir[3];
    float cutoff;
};
```

Figure 3: Estrutura de dados 'Light'

```
enum LightType {
    POINT,
    DIRECTIONAL,
    SPOTLIGHT
};
```

Figure 4: Enum 'LightType'

À estrutura *Model* foram adicionados 3 atributos e foram removidos 2:

- adicionou-se 3 **GLuint** - *vboN*, *vboT* e *texID* que definem, respetivamente, o índice do vbo das normais, o índice do vbo das texturas e o índice da textura;
- removeu-se os atributos *faces* e *numVertices* pois já não serviam qualquer propósito;

```

struct Model {
    string type; //Define a primitiva que representa o modelo
    vector<int> dividers; //Guarda onde começa cada face do modelo
    vector<string> faces; //Guarda o tipo de face do modelo
    GLuint vbo; //Index do vbo
    GLuint vboN; //Index vbo das normais
    GLuint vboT; //Index vbo das texturas
    GLuint texID; //Index da textura do modelo
};

```

Figure 5: Estrutura de dados 'Model'

À estrutura *File* foram adicionados 2 atributos:

- um **Color** - *color* que define a cor do modelo;
- uma **string** - *texture* que guarda o nome do ficheiro onde está a texture do modelo;

```

struct File{
    string file, texture;
    float dayPeriod = 0;
    Model modelo;
    Color color;
};

```

Figure 6: Estrutura de dados 'File'

3.2. Parser do ficheiro 3d

Uma vez que se alterou o formato dos ficheiros 3d, foi também necessário alterar a forma como este é lido pela engine. Para tal efeito procedeu-se a separação de cada linha em 3 ou 2 sublinhas, separadas por ';', em que a primeira sublinha é uma componente da coordenada de posição, a segunda sublinha é uma componente da normal e em que a terceira sublinha, se existir, é uma componente da coordenada de textura.

```

if (f.is_open()) {
    vector<float> p;
    vector<float> n;
    vector<float> t;
    // Read each line from the file and store it in the
    // 'line' variable.
    while (getline(f, line)) {
        if (line == "cone" || line == "box" || line == "plane" || line == "sphere" || line == "torus" || line == "bezier") {
            modelo.type = line;
        }
        else if (line[0] == 'G') {
            modelo.faces.push_back(line);
            modelo.dividers.push_back(counter/3);
        }
        else if (line != "") {
            int start = 0, end, c = 0;
            while ((end = line.find(';', start)) != string::npos) {
                if (c == 0) {
                    p.push_back(stof(line.substr(start, end - start)));
                }
                else if (c == 1) {
                    n.push_back(stof(line.substr(start, end - start)));
                }
                start = end + 1;
                c++;
            }
            if (c == 1) n.push_back(stof(line.substr(start)));
            else t.push_back(stof(line.substr(start)));
            counter++;
        }
    }

    // Close the file stream once all lines have been
    // read.
    f.close();
    modelo.vbo = buffer;
    modelo.vboN = buffer;
    modelo.vboT = buffer;
}

```

Figure 7: Parser do ficheiro 3d

Para guardar estes novos dados, é criado mais dois buffers por modelo onde são guardados as normais e as coordenadas das texturas.

3.3. Setup das luzes

Por cada luz introduzida no XML, criou-se uma luz no programa com as informações contidas nos ficheiros XML. As luzes foram desenhadas antes da configuração da câmara. Há de ressaltar que mesmo que o ficheiro XML não contenha nenhuma luz, os objetos serão observáveis, uma vez que existe uma componente que luz ambiente que foi introduzida por default.

3.4. Setup das cores

Para os objetos que continham cores, tivemos de configurar várias propriedades de material e luz para posterior renderização gráfica. Esses parâmetros de setup foram retirados dos ficheiros XML e continham parâmetros como as componentes de luz difusa, especular e emissiva e o brilho.

4. Sistema Solar

Para esta fase criamos uma sistema solar em que todos os planetas e satélites tem texturas.

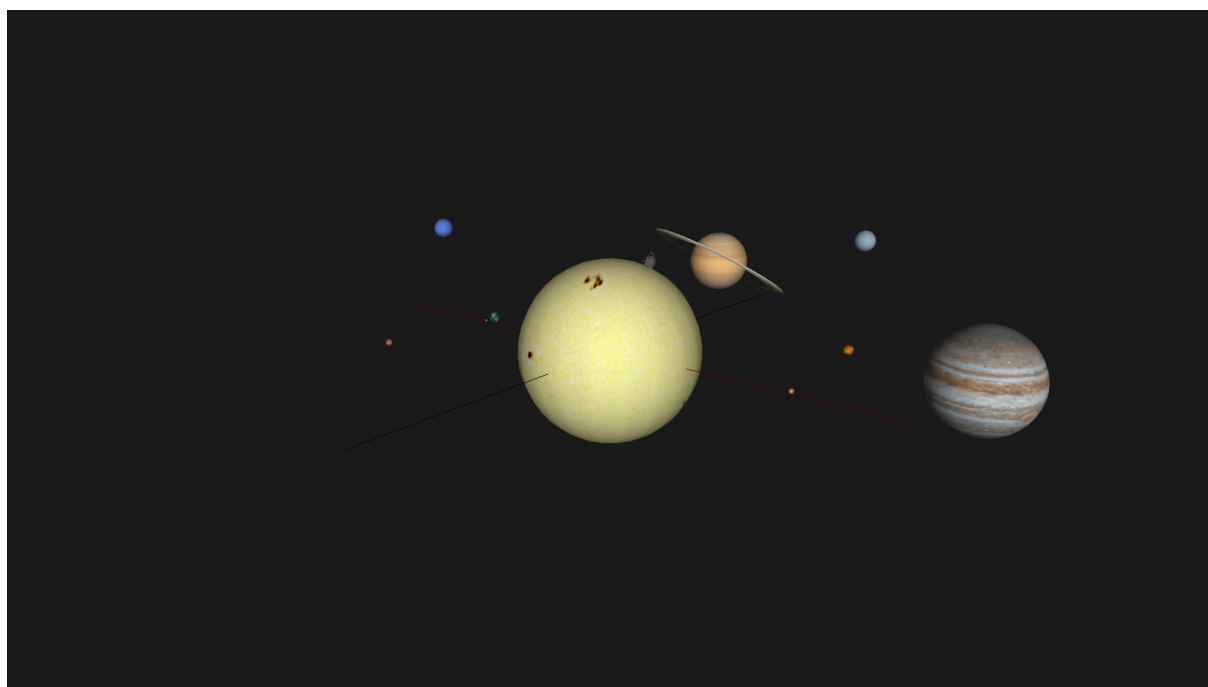


Figure 8: Sistema Solar

5. Conclusão

Para concluirmos esta última fase do projeto, foi necessário testar as nossas capacidades que envolvem a iluminação, cores e texturas para elevar significativamente o realismo e a qualidade do sistema solar. Para isto, foi também crucial aprofundar os nossos conhecimentos base, com o objetivo de reformatar e adaptar as partes do projeto feitas nas partes anteriores, para serem capazes de incluir as novas implementações.

Chegando agora ao fim do projeto, e tendo em conta o processo de desenvolvimento do mesmo, podemos dizer que apesar de ter sido um trabalho com dificuldades e obstáculos, foi também uma grande oportunidade de entender, por mais pouco que seja, o que é necessário no desenvolvimento gráfico.